Hognose: A simple modern mini-language

```
# Hello from Hognose!

int x = 3

if ( x < 5 ) {
    print("Hello, World!\n")
}
```

Here is a comprehensive design for "Hognose," including its philosophy, a formal grammar, and example code.

1. Core Philosophy of "Hognose"

- **Explicit is better than implicit:** All variables must be declared with a type. Function return types are mandatory.
  - No implicit down-casting, *promotion* is supported in numeric expressions.
- **Structure from syntax:** Code blocks and lexical scope are defined explicitly by curly braces `{...}`.
- **Simplicity:** The language has a minimal set of keywords, data types, and control flow structures to be Turing complete without being overwhelming.
- **Familiarity:** The syntax for expressions, function calls, and control flow is intentionally similar to C-family languages; comments are Python-esque.
- **Safety:** The strict type system and lack of pointers or manual memory management eliminate many common sources of bugs and undefined behavior.

2. Language Features

- **Data Types:**
  - `int`: 32-bit signed integer.
  - `float`: 64-bit IEEE 754 floating-point number.
  - `str`: A sequence of characters.
  - `bool`: A boolean value, with literals `true` and `false`.
- **Keywords:**
  - `int`, `float`, `str`, `bool`: Type specifiers.
  - `if`, `else`: Conditional control flow.
  - `while`: Looping control flow.
  - `true`, `false`: Boolean literals.
- **Syntax:**
  - One statement per line (no semicolons).
    - Otherwise, whitespace is "freeform", for example `x + 2` and `x+2` are both valid.
  - Lines may be continued by ending with `\`.
    - Parenthesized expressions may place parentheses on separate lines without needing to use escaped line break.

- Strings may not break across lines.
- Code blocks are enclosed in curly braces `{}`.
- Single-line comments start with `#`.
- Operators will follow standard C-style precedence.

- **Built-in Functions:**
  - To make the language usable, we provide a *very* small standard library.:
    - `print(expression)`: Prints the string representation of an expression's result to the console.
    - `str(expression)`: Converts the result of an expression to the `str` type.

---

3. Formal Grammar for "Hognose"

Here is a grammar in a simplified Backus-Naur Form (BNF) style. This is suitable for use with parser generator tools like ANTLR or for writing a recursive descent parser by hand.

Program Structure

A program is a sequence of top-level declarations. In Hognose, this means statements.

```
program                ::= (statement | NEWLINE)*
```

Statements

```
statement              ::= leading_ident_statement
                         | if_statement
                         | while_statement
                         | variable_declaration

block_statement        ::= '{' NEWLINE* (statement | NEWLINE)* '}' NEWLINE

statement_body         ::= block_statement | statement

leading_ident_statement ::= IDENTIFIER ident_statement_suffix

ident_statement_suffix  ::= '=' expression NEWLINE
                          | function_call_suffix NEWLINE

variable_declaration    ::= type IDENTIFIER ('=' expression)? NEWLINE

if_statement                ::= 'if' '(' NEWLINE* expression NEWLINE* ')'
statement_body else_part

else_part              ::= 'else' statement_body
                         | ε
```

```
while_statement            ::= 'while' '(' NEWLINE* expression NEWLINE* ')'
statement_body
```

Expressions (with Operator Precedence)

The grammar is defined from lowest to highest precedence to handle parsing correctly.

```
expression              ::= logical_or_expr

logical_or_expr         ::= logical_and_expr ('||' logical_and_expr)*
logical_and_expr        ::= equality_expr ('&&' equality_expr)*
equality_expr           ::= comparison_expr (('==' | '!=') comparison_expr)*
comparison_expr      ::= additive_expr (('<' | '>' | '<=' | '>=') additive_expr)*
additive_expr        ::= multiplicative_expr (('+' | '-') multiplicative_expr)*
multiplicative_expr     ::= unary_expr (('*' | '/' | '%') unary_expr)*

unary_expr              ::= ('!' | '-') unary_expr
                          | primary_expr

primary_expr            ::= literal
                          | IDENTIFIER ident_expr_suffix
                          | '(' expression ')'
                          | cast_expression

cast_expression         ::= type '(' NEWLINE* expression NEWLINE* ')'

ident_expr_suffix       ::= function_call_suffix
                          | ε

function_call_suffix    ::= '(' NEWLINE* argument_list? NEWLINE* ')'

argument_list           ::= expression (',' NEWLINE* expression)*
```

Literals and Terminals

REs below are shown in Python `re` syntax, without enclosing quotes.

```
literal                 ::= LITERAL_BOOL
                          | LITERAL_FLOAT
                          | LITERAL_INT
                          | LITERAL_STR

type                    ::= bool|float|int|str
```

```
IDENTIFIER              ::= [a-zA-Z_][a-zA-Z0-9_]*
LITERAL_BOOL            ::= false|true
LITERAL_FLOAT           ::= [0-9]+\.[0-9]+
LITERAL_INT             ::= [0-9]+
LITERAL_STR             ::= "[^"\n]+"
NEWLINE                 ::= \s*\n
```

---

4. Type System and Semantics

Some notes about the semantics of the language:

1. **Scope:** Hognose uses lexical (static) scoping. A new scope is created by each `{}` block. A variable declared inside a block is not visible outside that block. Inner variables can "shadow" outer variables of the same name.
2. **Type Checking:** This is a key part of your project.
   - **Variables:** A variable must be declared before it is used. The type of the value in an assignment must match the declared type of the variable.
   - **Operators:**
     - `+`, `-`, `*`, `/`, `%`: Operate on `int` and `float`. If one operand is `float`, the other is promoted to `float` and the result is `float`.
     - `+`: Is also defined for `str` concatenation. `str + str -> str`.
     - `==`, `!=`: Can compare any two operands of the *same* type (or promoted types).
     - `<`, `>`, `<=`, `>=`: Operate on `int` and `float`.
     - `&&`, `||`, `!`: Operate only on `bool` types.
     - Any other combination (e.g., `int + str`, `bool * int`) is a compile-time type error.
   - **Control Flow:** The condition expression in an `if` or `while` statement *must* evaluate to a `bool`.

---

5. Example "Hognose" Code

Here are a few examples to illustrate the language's look and feel.

Example 1: Hello, World!

```
# This is a simple program to demonstrate variable declaration and printing.

str message = "Hello, World!"
print(message)

int year = 2025
# The str() function allows + to concatenate the int as a str
print("The year is: " + str(year))
```

Example 2: Line continuation

```
int first = 10
int second = 20
int third = 30

# This is parsed as a single statement: result = first + second + third
int result = first + second + \
                third

print("The very long calculation result is: " + \
        str(result))
```

Example 3: Variables

```
# Add two integers.

int x = 10
int y = 25
int sum = x + y

print("The sum is: ")
print(sum)
```

Example 4: Looping and Conditionals (Factorial)

```
# A program to calculate the factorial of a number using a while loop.
int n = 7
# ---------------------------
int result = 1
int i      = 1
while (i <= n) {
   result = result * i
   i = i + 1
}
# ---------------------------
print("Factorial of " + str(n) + " is " + str(result))
```